

Chapter 7. Reliable Data Delivery

Reliability is a property of a system—not of a single component—so when we are talking about the reliability guarantees of Apache Kafka, we will need to keep the entire system and its use cases in mind. When it comes to reliability, the systems that integrate with Kafka are as important as Kafka itself. And because reliability is a system concern, it cannot be the responsibility of just one person. Everyone—Kafka administrators, Linux administrators, network and storage administrators, and the application developers—must work together to build a reliable system.

Apache Kafka is very flexible about reliable data delivery. We understand that Kafka has many use cases, from tracking clicks in a website to credit card payments. Some of the use cases require utmost reliability, while others prioritize speed and simplicity over reliability. Kafka was written to be configurable enough, and its client API flexible enough, to allow all kinds of reliability trade-offs.

Because of its flexibility, it is also easy to accidentally shoot ourselves in the foot when using Kafka—believing that our system is reliable when in fact it is not. In this chapter, we will start by talking about different kinds of reliability and what they mean in the context of Apache Kafka. Then we will talk about Kafka’s replication mechanism and how it contributes to the reliability of the system. We will then discuss Kafka’s brokers and topics and how they should be configured for different use cases. Then we will discuss the clients, producer, and consumer, and how they should be used in different reliability scenarios. Last, we will discuss the topic of validating the system reliability, because it is not enough to believe a system is reliable—the assumption must be thoroughly tested.

Reliability Guarantees

When we talk about reliability, we usually talk in terms of *guarantees*, which are the behaviors a system is guaranteed to preserve under differ-

ent circumstances.

Probably the best-known reliability guarantee is ACID, which is the standard reliability guarantee that relational databases universally support. ACID stands for *atomicity*, *consistency*, *isolation*, and *durability*. When a vendor explains that their database is ACID compliant, it means the database guarantees certain behaviors regarding transaction behavior.

Those guarantees are the reason people trust relational databases with their most critical applications—they know exactly what the system promises and how it will behave in different conditions. They understand the guarantees and can write safe applications by relying on those guarantees.

Understanding the guarantees Kafka provides is critical for those seeking to build reliable applications. This understanding allows the developers of the system to figure out how it will behave under different failure conditions. So, what does Apache Kafka guarantee?

- Kafka provides order guarantee of messages in a partition. If message B was written after message A, using the same producer in the same partition, then Kafka guarantees that the offset of message B will be higher than message A, and that consumers will read message B after message A.
- Produced messages are considered “committed” when they were written to the partition on all its in-sync replicas (but not necessarily flushed to disk). Producers can choose to receive acknowledgments of sent messages when the message was fully committed, when it was written to the leader, or when it was sent over the network.
- Messages that are committed will not be lost as long as at least one replica remains alive.
- Consumers can only read messages that are committed.

These basic guarantees can be used while building a reliable system, but in themselves, they don’t make the system fully reliable. There are trade-offs involved in building a reliable system, and Kafka was built to allow administrators and developers to decide how much reliability they need by providing configuration parameters that allow controlling these trade-offs. The trade-offs usually involve how important it is to reliably and

consistently store messages versus other important considerations, such as availability, high throughput, low latency, and hardware costs.

We next review Kafka's replication mechanism, introduce terminology, and discuss how reliability is built into Kafka. After that, we go over the configuration parameters we just mentioned.

Replication

Kafka's replication mechanism, with its multiple replicas per partition, is at the core of all of Kafka's reliability guarantees. Having a message written in multiple replicas is how Kafka provides durability of messages in the event of a crash.

We explained Kafka's replication mechanism in depth in [Chapter 6](#), but let's recap the highlights here.

Each Kafka topic is broken down into *partitions*, which are the basic data building blocks. A partition is stored on a single disk. Kafka guarantees the order of events within a partition, and a partition can be either online (available) or offline (unavailable). Each partition can have multiple replicas, one of which is a designated leader. All events are produced to the leader replica and are usually consumed from the leader replica as well. Other replicas just need to stay in sync with the leader and replicate all the recent events on time. If the leader becomes unavailable, one of the in-sync replicas becomes the new leader (there is an exception to this rule, which we discussed in [Chapter 6](#)).

A replica is considered in sync if it is the leader for a partition, or if it is a follower that:

- Has an active session with ZooKeeper—meaning that it sent a heart-beat to ZooKeeper in the last 6 seconds (configurable).
- Fetched messages from the leader in the last 10 seconds (configurable).
- Fetched the most recent messages from the leader in the last 10 seconds. That is, it isn't enough that the follower is still getting messages from the leader; it must have had no lag at least once in the last 10 seconds (configurable).

If a replica loses connection to ZooKeeper, stops fetching new messages, or falls behind and can't catch up within 10 seconds, the replica is considered out of sync. An out-of-sync replica gets back into sync when it connects to ZooKeeper again and catches up to the most recent message written to the leader. This usually happens quickly after a temporary network glitch is healed but can take a while if the broker the replica is stored on was down for a longer period of time.

OUT-OF-SYNC REPLICAS

In older versions of Kafka, it was not uncommon to see one or more replicas rapidly flip between in-sync and out-of-sync status. This was a sure sign that something was wrong with the cluster. A relatively common cause was a large maximum request size and large JVM heap that required tuning to prevent long garbage collection pauses that would cause the broker to temporarily disconnect from ZooKeeper. These days the problem is very rare, especially when using Apache Kafka release 2.5.0 and higher with its default configurations for ZooKeeper connection timeout and maximum replica lag. The use of JVM version 8 and above (now the minimum version supported by Kafka) with [G1 garbage collector](#) helped curb this problem, although tuning may still be required for large messages. Generally speaking, Kafka's replication protocol became significantly more reliable in the years since the first edition of the book was published. For details on the evolution of Kafka's replication protocol, refer to Jason Gustafson's excellent talk, ["Hardening Apache Kafka Replication"](#), and Gwen Shapira's overview of Kafka improvements, ["Please Upgrade Apache Kafka Now"](#).

An in-sync replica that is slightly behind can slow down producers and consumers—since they wait for all the in-sync replicas to get the message before it is *committed*. Once a replica falls out of sync, we no longer wait for it to get messages. It is still behind, but now there is no performance impact. The catch is that with fewer in-sync replicas, the effective replication factor of the partition is lower, and therefore there is a higher risk for downtime or data loss.

In the next section, we will look at what this means in practice.

Broker Configuration

There are three configuration parameters in the broker that change Kafka's behavior regarding reliable message storage. Like many broker configuration variables, these can apply at the broker level, controlling configuration for all topics in the system, and at the topic level, controlling behavior for a specific topic.

Being able to control reliability trade-offs at the topic level means that the same Kafka cluster can be used to host reliable and nonreliable topics. For example, at a bank, the administrator will probably want to set very reliable defaults for the entire cluster but make an exception to the topic that stores customer complaints where some data loss is acceptable.

Let's look at these configuration parameters one by one and see how they affect the reliability of message storage in Kafka and the trade-offs involved.

Replication Factor

The topic-level configuration is `replication.factor`. At the broker level, we control the `default.replication.factor` for automatically created topics.

Until this point in the book, we have assumed that topics had a replication factor of three, meaning that each partition is replicated three times on three different brokers. This was a reasonable assumption, as this is Kafka's default, but this is a configuration that users can modify. Even after a topic exists, we can choose to add or remove replicas and thereby modify the replication factor using Kafka's replica assignment tool.

A replication factor of N allows us to lose $N-1$ brokers while still being able to read and write data to the topic. So a higher replication factor leads to higher availability, higher reliability, and fewer disasters. On the flip side, for a replication factor of N , we will need at least N brokers and we will store N copies of the data, meaning we will need N times as much disk space. We are basically trading availability for hardware.

So how do we determine the right number of replicas for a topic? There are a few key considerations:

Availability

A partition with just one replica will become unavailable even during a routine restart of a single broker. The more replicas we have, the higher availability we can expect.

Durability

Each replica is a copy of all the data in a partition. If a partition has a single replica and the disk becomes unusable for any reason, we've lost all the data in the partition. With more copies, especially on different storage devices, the probability of losing all of them is reduced.

Throughput

With each additional replica, we multiply the inter-broker traffic. If we produce to a partition at a rate of 10 MBps, then a single replica will not generate any replication traffic. If we have 2 replicas, then we'll have 10 MBps replication traffic, with 3 replicas it will be 20 MBps, and with 5 replicas it will be 40 MBps. We need to take this into account when planning the cluster size and capacity.

End-to-end latency

Each produced record has to be replicated to all in-sync replicas before it is available for consumers. In theory, with more replicas, there is higher probability that one of these replicas is a bit slow and therefore will slow the consumers down. In practice, if one broker becomes slow for any reason, it will slow down every client that tries using it, regardless of replication factor.

Cost

This is the most common reason for using a replication factor lower than 3 for noncritical data. The more replicas we have of our data, the higher the storage and network costs. Since many storage systems already replicate each block 3 times, it sometimes makes sense to reduce costs by configuring Kafka with a replication factor of 2. Note that this will still reduce availability compared to a repli-

cation factor of 3, but durability will be guaranteed by the storage device.

Placement of replicas is also very important. Kafka will always make sure each replica for a partition is on a separate broker. In some cases, this is not safe enough. If all replicas for a partition are placed on brokers that are on the same rack, and the top-of-rack switch misbehaves, we will lose availability of the partition regardless of the replication factor. To protect against rack-level misfortune, we recommend placing brokers in multiple racks and using the `broker.rack` broker configuration parameter to configure the rack name for each broker. If rack names are configured, Kafka will make sure replicas for a partition are spread across multiple racks in order to guarantee even higher availability. When running Kafka in cloud environments, it is common to consider availability zones as separate racks. In [Chapter 6](#), we provided details on how Kafka places replicas on brokers and racks.

Unclean Leader Election

This configuration is only available at the broker (and in practice, cluster-wide) level. The parameter name is `unclean.leader.election.enable`, and by default it is set to `false`.

As explained earlier, when the leader for a partition is no longer available, one of the in-sync replicas will be chosen as the new leader. This leader election is “clean” in the sense that it guarantees no loss of committed data—by definition, committed data exists on all in-sync replicas.

But what do we do when no in-sync replica exists except for the leader that just became unavailable?

This situation can happen in one of two scenarios:

- The partition had three replicas, and the two followers became unavailable (let’s say two brokers crashed). In this situation, as producers continue writing to the leader, all the messages are acknowledged and committed (since the leader is the one and only in-sync replica). Now let’s say that the leader becomes unavailable (oops, another broker crash). In this scenario, if one of the out-of-sync followers starts first,

we have an out-of-sync replica as the only available replica for the partition.

- The partition had three replicas, and due to network issues, the two followers fell behind so that even though they are up and replicating, they are no longer in sync. The leader keeps accepting messages as the only in-sync replica. Now if the leader becomes unavailable, there are only out-of-sync replicas available to become leaders.

In both these scenarios, we need to make a difficult decision:

- If we don't allow the out-of-sync replica to become the new leader, the partition will remain offline until we bring the old leader (and the last in-sync replica) back online. In some cases (e.g., memory chip needs replacement), this can take many hours.
- If we do allow the out-of-sync replica to become the new leader, we are going to lose all messages that were written to the old leader while that replica was out of sync and also cause some inconsistencies in consumers. Why? Imagine that while replicas 0 and 1 were not available, we wrote messages with offsets 100–200 to replica 2 (then the leader). Now replica 2 is unavailable and replica 0 is back online. Replica 0 only has messages 0–100 but not 100–200. If we allow replica 0 to become the new leader, it will allow producers to write new messages and allow consumers to read them. So, now the new leader has completely new messages 100–200. First, let's note that some consumers may have read the old messages 100–200, some consumers got the new 100–200, and some got a mix of both. This can lead to pretty bad consequences when looking at things like downstream reports. In addition, replica 2 will come back online and become a follower of the new leader. At that point, it will delete any messages it got that don't exist on the current leader. Those messages will not be available to any consumer in the future.

In summary, if we allow out-of-sync replicas to become leaders, we risk data loss and inconsistencies. If we don't allow them to become leaders, we face lower availability as we must wait for the original leader to become available before the partition is back online.

By default, `unclean.leader.election.enable` is set to false, which will not allow out-of-sync replicas to become leaders. This is the safest option since it provides the best guarantees against data loss. It does mean

that in the extreme unavailability scenarios that we described previously, some partitions will remain unavailable until manually recovered. It is always possible for an administrator to look at the situation, decide to accept the data loss in order to make the partitions available, and switch this configuration to true before starting the cluster. Just don't forget to turn it back to false after the cluster recovered.

Minimum In-Sync Replicas

Both the topic and the broker-level configuration are called `min.insync.replicas`.

As we've seen, there are cases where even though we configured a topic to have three replicas, we may be left with a single in-sync replica. If this replica becomes unavailable, we may have to choose between availability and consistency. This is never an easy choice. Note that part of the problem is that, per Kafka reliability guarantees, data is considered committed when it is written to all in-sync replicas, even when `all` means just one replica and the data could be lost if that replica is unavailable.

When we want to be sure that committed data is written to more than one replica, we need to set the minimum number of in-sync replicas to a higher value. If a topic has three replicas and we set `min.insync.replicas` to `2`, then producers can only write to a partition in the topic if at least two out of the three replicas are in sync.

When all three replicas are in sync, everything proceeds normally. This is also true if one of the replicas becomes unavailable. However, if two out of three replicas are not available, the brokers will no longer accept produce requests. Instead, producers that attempt to send data will receive `NotEnoughReplicasException`. Consumers can continue reading existing data. In effect, with this configuration, a single in-sync replica becomes read-only. This prevents the undesirable situation where data is produced and consumed, only to disappear when an election occurs. In order to recover from this read-only situation, we must make one of the two unavailable partitions available again (maybe restart the broker) and wait for it to catch up and get in sync.

Keeping Replicas In Sync

As mentioned earlier, out-of-sync replicas decrease the overall reliability, so it is important to avoid these as much as possible. We also explained that a replica can become out of sync in one of two ways: either it loses connectivity to ZooKeeper or it fails to keep up with the leader and builds up a replication lag. Kafka has two broker configurations that control the sensitivity of the cluster to these two conditions.

`zookeeper.session.timeout.ms` is the time interval during which a Kafka broker can stop sending heartbeats to ZooKeeper without ZooKeeper considering the broker dead and removing it from the cluster. In version 2.5.0, this value was increased from 6 seconds to 18 seconds, in order to increase the stability of Kafka clusters in cloud environments where network latencies show higher variance. In general, we want this time to be high enough to avoid random flapping caused by garbage collection or network conditions, but still low enough to make sure brokers that are actually frozen will be detected in a timely manner.

If a replica did not fetch from the leader or did not catch up to the latest messages on the leader for longer than `replica.lag.time.max.ms`, it will become out of sync. This was increased from 10 seconds to 30 seconds in release 2.5.0 to improve resilience of the cluster and avoid unnecessary flapping. Note that this higher value also impacts maximum latency for the consumer—with the higher value it can take up to 30 seconds until a message arrives to all replicas and the consumers are allowed to consume it.

Persisting to Disk

We've mentioned a few times that Kafka will acknowledge messages that were not persisted to disk, depending just on the number of replicas that received the message. Kafka will flush messages to disk when rotating segments (by default 1 GB in size) and before restarts but will otherwise rely on Linux page cache to flush messages when it becomes full. The idea behind this is that having three machines in separate racks or availability zones, each with a copy of the data, is safer than writing the messages to disk on the leader, because simultaneous failures on two different racks or zones are so unlikely. However, it is possible to configure the

brokers to persist messages to disk more frequently. The configuration parameter `flush.messages` allows us to control the maximum number of messages not synced to disk, and `flush.ms` allows us to control the frequency of syncing to disk. Before using this feature, it is worth reading [how `fsync` impacts Kafka's throughput and how to mitigate its drawbacks](#).

Using Producers in a Reliable System

Even if we configure the brokers in the most reliable configuration possible, the system as a whole can still potentially lose data if we don't configure the producers to be reliable as well.

Here are two example scenarios to demonstrate this:

- We configured the brokers with three replicas, and unclean leader election is disabled. So we should never lose a single message that was committed to the Kafka cluster. However, we configured the producer to send messages with `acks=1`. We sent a message from the producer, and it was written to the leader but not yet to the in-sync replicas. The leader sent back a response to the producer saying, "Message was written successfully" and immediately crashes before the data was replicated to the other replicas. The other replicas are still considered in sync (remember that it takes a while before we declare a replica out of sync), and one of them will become the leader. Since the message was not written to the replicas, it was lost. But the producing application thinks it was written successfully. The system is consistent because no consumer saw the message (it was never committed because the replicas never got it), but from the producer perspective, a message was lost.
- We configured the brokers with three replicas, and unclean leader election is disabled. We learned from our mistakes and started producing messages with `acks=all`. Suppose that we are attempting to write a message to Kafka, but the leader for the partition we are writing to just crashed and a new one is still getting elected. Kafka will respond with "Leader not Available." At this point, if the producer doesn't handle the error correctly and doesn't retry until the write is successful, the message may be lost. Once again, this is not a broker reliability issue because the broker never got the message; and it is not a

consistency issue because the consumers never got the message either. But if producers don't handle errors correctly, they may cause message loss.

As the examples show, there are two important things that everyone who writes applications that produce to Kafka must pay attention to:

- Use the correct `acks` configuration to match reliability requirements
- Handle errors correctly both in configuration and in code

We discussed producer configuration in depth in [Chapter 3](#), but let's go over the important points again.

Send Acknowledgments

Producers can choose between three different acknowledgment modes:

`acks=0`

This means that a message is considered to be written successfully to Kafka if the producer managed to send it over the network. We will still get errors if the object we are sending cannot be serialized or if the network card failed, but we won't get any error if the partition is offline, a leader election is in progress, or even if the entire Kafka cluster is unavailable. Running with `acks=0` has low produce latency (which is why we see a lot of benchmarks with this configuration), but it will not improve end-to-end latency (remember that consumers will not see messages until they are replicated to all available replicas).

`acks=1`

This means that the leader will send either an acknowledgment or an error the moment it gets the message and writes it to the partition data file (but not necessarily synced to disk). We can lose data if the leader shuts down or crashes and some messages that were successfully written to the leader and acknowledged were not replicated to the followers before the crash. With this configuration, it is also possible to write to the leader faster than it can replicate messages and end up with under-replicated partitions, since

the leader will acknowledge messages from the producer before replicating them.

`acks=all`

This means that the leader will wait until all in-sync replicas get the message before sending back an acknowledgment or an error. In conjunction with the `min.insync.replicas` configuration on the broker, this lets us control how many replicas get the message before it is acknowledged. This is the safest option—the producer won’t stop trying to send the message before it is fully committed. This is also the option with the longest producer latency—the producer waits for all in-sync replicas to get all the messages before it can mark the message batch as “done” and carry on.

Configuring Producer Retries

There are two parts to handling errors in the producer: the errors that the producers handle automatically for us and the errors that we, as developers using the producer library, must handle.

The producer can handle *retriable* errors. When the producer sends messages to a broker, the broker can return either a success or an error code. Those error codes belong to two categories—errors that can be resolved after retrying and errors that won’t be resolved. For example, if the broker returns the error code `LEADER_NOT_AVAILABLE`, the producer can try sending the message again—maybe a new broker was elected and the second attempt will succeed. This means that `LEADER_NOT_AVAILABLE` is a *retriable* error. On the other hand, if a broker returns an `INVALID_CONFIG` exception, trying the same message again will not change the configuration. This is an example of a *nonretriable error*.

In general, when our goal is to never lose a message, our best approach is to configure the producer to keep trying to send the messages when it encounters a retriable error. And the best approach to retries, as recommended in [Chapter 3](#), is to leave the number of retries at its current default (`MAX_INT`, or effectively infinite) and use `delivery.timeout.ms` to configure the maximum amount of time we are willing to wait until giving up on sending a message—the producer will retry sending the message as many times as possible within this time interval.

Retrying to send a failed message includes a risk that both messages were successfully written to the broker, leading to duplicates. Retries and careful error handling can guarantee that each message will be stored *at least once*, but not *exactly once*. Using `enable.idempotence=true` will cause the producer to include additional information in its records, which brokers will use to skip duplicate messages caused by retries. In [Chapter 8](#), we discuss in detail how and when this works.

Additional Error Handling

Using the built-in producer retries is an easy way to correctly handle a large variety of errors without loss of messages, but as developers, we must still be able to handle other types of errors. These include:

- Nonretriable broker errors, such as errors regarding message size, authorization errors, etc.
- Errors that occur before the message was sent to the broker—for example, serialization errors
- Errors that occur when the producer exhausted all retry attempts or when the available memory used by the producer is filled to the limit due to using all of it to store messages while retrying
- Timeouts

In [Chapter 3](#) we discussed how to write error handlers for both sync and async message-sending methods. The content of these error handlers is specific to the application and its goals—do we throw away “bad messages”? Log errors? Stop reading messages from the source system? Apply back pressure to the source system to stop sending messages for a while? Store these messages in a directory on the local disk? These decisions depend on the architecture and the product requirements. Just note that if all the error handler is doing is retrying to send the message, then we’ll be better off relying on the producer’s retry functionality.

Using Consumers in a Reliable System

Now that we have learned how to produce data while taking Kafka’s reliability guarantees into account, it is time to see how to consume data.

As we saw in the first part of this chapter, data is only available to consumers after it has been committed to Kafka—meaning it was written to all in-sync replicas. This means that consumers get data that is guaranteed to be consistent. The only thing consumers are left to do is make sure they keep track of which messages they’ve read and which messages they haven’t. This is key to not losing messages while consuming them.

When reading data from a partition, a consumer is fetching a batch of messages, checking the last offset in the batch, and then requesting another batch of messages starting from the last offset received. This guarantees that a Kafka consumer will always get new data in correct order without missing any messages.

When a consumer stops, another consumer needs to know where to pick up the work—what was the last offset that the previous consumer processed before it stopped? The “other” consumer can even be the original one after a restart. It doesn’t really matter—some consumer is going to pick up consuming from that partition, and it needs to know at which offset to start. This is why consumers need to “commit” their offsets. For each partition it is consuming, the consumer stores its current location, so if or another consumer will know where to continue after a restart. The main way consumers can lose messages is when committing offsets for events they’ve read but haven’t completely processed yet. This way, when another consumer picks up the work, it will skip those messages and they will never get processed. This is why paying careful attention to when and how offsets get committed is critical.

COMMITTED MESSAGES VERSUS COMMITTED OFFSETS

This is different from a *committed message*, which, as discussed previously, is a message that was written to all in-sync replicas and is available to consumers. *Committed offsets* are offsets the consumer sent to Kafka to acknowledge that it received and processed all the messages in a partition up to this specific offset.

In [Chapter 4](#), we discussed the Consumer API in detail and covered the many methods for committing offsets. Here we will cover some important considerations and choices, but refer back to [Chapter 4](#) for details on using the APIs.

Important Consumer Configuration Properties for Reliable Processing

There are four consumer configuration properties that are important to understand in order to configure our consumer for a desired reliability behavior.

The first is `group.id`, as explained in great detail in [Chapter 4](#). The basic idea is that if two consumers have the same group ID and subscribe to the same topic, each will be assigned a subset of the partitions in the topic and will therefore only read a subset of the messages individually (but all the messages will be read by the group as a whole). If we need a consumer to see, on its own, every single message in the topics it is subscribed to, it will need a unique `group.id`.

The second relevant configuration is `auto.offset.reset`. This parameter controls what the consumer will do when no offsets were committed (e.g., when the consumer first starts) or when the consumer asks for offsets that don't exist in the broker ([Chapter 4](#) explains how this can happen). There are only two options here. If we choose `earliest`, the consumer will start from the beginning of the partition whenever it doesn't have a valid offset. This can lead to the consumer processing a lot of messages twice, but it guarantees to minimize data loss. If we choose `latest`, the consumer will start at the end of the partition. This minimizes duplicate processing by the consumer but almost certainly leads to some messages getting missed by the consumer.

The third relevant configuration is `enable.auto.commit`. This is a big decision: are we going to let the consumer commit offsets for us based on schedule, or are we planning on committing offsets manually in our code? The main benefit of automatic offset commits is that it's one less thing to worry about when using consumers in our application. When we do all the processing of consumed records within the consumer poll loop, then the automatic offset commit guarantees we will never accidentally commit an offset that we didn't process. The main drawbacks of automatic offset commits is that we have no control over the number of duplicate records the application may process because it was stopped after processing some records but before the automated commit kicked in. When the application has more complex processing, such as passing records to an-

other thread to process in the background, there is no choice but to use manual offset commit since the automatic commit may commit offsets for records the consumer has read but perhaps has not processed yet.

The fourth relevant configuration, `auto.commit.interval.ms`, is tied to the third. If we choose to commit offsets automatically, this configuration lets us configure how frequently they will be committed. The default is every five seconds. In general, committing more frequently adds overhead but reduces the number of duplicates that can occur when a consumer stops.

While not directly related to reliable data processing, it is difficult to consider a consumer reliable if it frequently stops consuming in order to rebalance. [Chapter 4](#) includes advice on how to configure consumers to minimize unnecessary rebalancing and to minimize pauses while rebalancing.

Explicitly Committing Offsets in Consumers

If we decide we need more control and choose to commit offsets manually, we need to be concerned about correctness and performance implications.

We will not go over the mechanics and APIs involved in committing offsets here, since they were covered in great depth in [Chapter 4](#). Instead, we will review important considerations when developing a consumer to handle data reliably. We'll start with the simple and perhaps obvious points and move on to more complex patterns.

Always commit offsets after messages were processed

If we do all the processing within the poll loop and don't maintain state between poll loops (e.g., for aggregation), this should be easy. We can use the auto-commit configuration, commit offset at the end of the poll loop, or commit offset inside the loop at a frequency that balances requirements for both overhead and lack of duplicate processing. If there are additional threads or stateful processing involved, this becomes more complex, especially since the consumer object is not thread safe. In [Chapter 4](#), we discussed how this can be done and provided references with additional examples.

Commit frequency is a trade-off between performance and number of duplicates in the event of a crash

Even in the simplest case where we do all the processing within the poll loop and don't maintain state between poll loops, we can choose to commit multiple times within a loop or choose to only commit every several loops. Committing has significant performance overhead. It is similar to produce with `acks=all`, but all offset commits of a single consumer group are produced to the same broker, which can become overloaded. The commit frequency has to balance requirements for performance and lack of duplicates. Committing after every message should only ever be done on very low-throughput topics.

Commit the right offsets at the right time

A common pitfall when committing in the middle of the poll loop is accidentally committing the last offset read when polling and not the offset after the last offset processed. Remember that it is critical to always commit offsets for messages after they were processed—committing offsets for messages read but not processed can lead to the consumer missing messages. [Chapter 4](#) has examples that show how to do just that.

Rebalances

When designing an application, we need to remember that consumer rebalances will happen, and we need to handle them properly. [Chapter 4](#) contains a few examples. This usually involves committing offsets before partitions are revoked and cleaning any state the application maintains when it is assigned new partitions.

Consumers may need to retry

In some cases, after calling poll and processing records, some records are not fully processed and will need to be processed later. For example, we may try to write records from Kafka to a database but find that the database is not available at that moment and we need to retry later. Note that unlike traditional pub/sub messaging systems, Kafka consumers commit offsets and do not “ack” individual messages. This means that if we failed to process record #30 and succeeded in processing record #31, we should not commit offset #31—this would result in marking as processed all the

records up to #31 including #30, which is usually not what we want. Instead, try following one of the following two patterns.

One option when we encounter a retrievable error is to commit the last record we processed successfully. We'll then store the records that still need to be processed in a buffer (so the next poll won't override them), use the consumer `pause()` method to ensure that additional polls won't return data, and keep trying to process the records.

A second option when encountering a retrievable error is to write it to a separate topic and continue. A separate consumer group can be used to handle retries from the retry topic, or one consumer can subscribe to both the main topic and to the retry topic but pause the retry topic between retries. This pattern is similar to the dead-letter-queue system used in many messaging systems.

Consumers may need to maintain state

In some applications, we need to maintain state across multiple calls to poll. For example, if we want to calculate moving average, we'll want to update the average after every time we poll Kafka for new messages. If our process is restarted, we will need to not just start consuming from the last offset, but we'll also need to recover the matching moving average. One way to do this is to write the latest accumulated value to a "results" topic at the same time the application is committing the offset. This means that when a thread is starting up, it can pick up the latest accumulated value when it starts and pick up right where it left off. In [Chapter 8](#), we discuss how an application can write results and commit offsets in a single transaction. In general, this is a rather complex problem to solve, and we recommend looking at a library like Kafka Streams or Flink, which provides high-level DSL-like APIs for aggregation, joins, windows, and other complex analytics.

Validating System Reliability

Once we have gone through the process of figuring out our reliability requirements, configuring the brokers, configuring the clients, and using the APIs in the best way for our use case, we can just relax and run everything in production, confident that no event will ever be missed, right?

We recommend doing some validation first and suggest three layers of validation: validate the configuration, validate the application, and monitor the application in production. Let's look at each of these steps and see what we need to validate and how.

Validating Configuration

It is easy to test the broker and client configuration in isolation from the application logic, and it is recommended to do so for two reasons:

- It helps to test if the configuration we've chosen can meet our requirements.
- It is a good exercise to reason through the expected behavior of the system.

Kafka includes two important tools to help with this validation. The `org.apache.kafka.tools` package includes `VerifiableProducer` and `VerifiableConsumer` classes. These can run as command-line tools or be embedded in an automated testing framework.

The idea is that the verifiable producer produces a sequence of messages containing numbers from 1 to a value we choose. We can configure the verifiable producer the same way we configure our own producer, setting the right number of `acks`, `retries`, `delivery.timeout.ms`, and rate at which the messages will be produced. When we run it, it will print success or error for each message sent to the broker, based on the `acks` received. The verifiable consumer performs the complementary check. It consumes events (usually those produced by the verifiable producer) and prints out the events it consumed in order. It also prints information regarding commits and rebalances.

It is important to consider which tests we want to run. For example:

- Leader election: what happens if we kill the leader? How long does it take the producer and consumer to start working as usual again?
- Controller election: how long does it take the system to resume after a restart of the controller?
- Rolling restart: can we restart the brokers one by one without losing any messages?

- Unclean leader election test: what happens when we kill all the replicas for a partition one by one (to make sure each goes out of sync) and then start a broker that was out of sync? What needs to happen in order to resume operations? Is this acceptable?

Then we pick a scenario, start the verifiable producer, start the verifiable consumer, and run through the scenario—for example, kill the leader of the partition we are producing data into. If we expected a short pause and then everything to resume normally with no message loss, we need to make sure the number of messages produced by the producer and the number of messages consumed by the consumer match.

The Apache Kafka source repository includes an [extensive test suite](#). Many of the tests in the suite are based on the same principle and use the verifiable producer and consumer to make sure rolling upgrades work.

Validating Applications

Once we are sure the broker and client configuration meet our requirements, it is time to test whether the application provides the guarantees we need. This will check things like custom error-handling code, offset commits, and rebalance listeners and similar places where the application logic interacts with Kafka’s client libraries.

Naturally, because application logic can vary considerably, there is only so much guidance we can provide on how to test it. We recommend integration tests for the application as part of any development process, and we recommend running tests under a variety of failure conditions:

- Clients lose connectivity to one of the brokers
- High latency between client and broker
- Disk full
- Hanging disk (also called “brown out”)
- Leader election
- Rolling restart of brokers
- Rolling restart of consumers
- Rolling restart of producers

There are many tools that can be used to introduce network and disk faults, and many are excellent, so we will not attempt to make specific

recommendations. Apache Kafka itself includes the [Trogdor test framework](#) for fault injection. For each scenario, we will have *expected behavior*, which is what we planned on seeing when we developed the application. Then we run the test to see what actually happens. For example, when planning for a rolling restart of consumers, we planned for a short pause as consumers rebalance and then continue consumption with no more than 1,000 duplicate values. Our test will show whether the way the application commits offsets and handles rebalances actually works this way.

Monitoring Reliability in Production

Testing the application is important, but it does not replace the need to continuously monitor production systems to make sure data is flowing as expected. [Chapter 12](#) will cover detailed suggestions on how to monitor the Kafka cluster, but in addition to monitoring the health of the cluster, it is important to also monitor the clients and the flow of data through the system.

Kafka's Java clients include JMX metrics that allow monitoring client-side status and events. For the producers, the two metrics most important for reliability are error-rate and retry-rate per record (aggregated). Keep an eye on those, since error or retry rates going up can indicate an issue with the system. Also monitor the producer logs for errors that occur while sending events that are logged at `WARN` level, and say something along the lines of "Got error produce response with correlation id 5689 on topic-partition [topic-1,3], retrying (two attempts left). Error: ..." When we see events with 0 attempts left, the producer is running out of retries. In [Chapter 3](#) we discussed how to configure `delivery.timeout.ms` and `retries` to improve the error handling in the producer and avoid running out of retries prematurely. Of course, it is always better to solve the problem that caused the errors in the first place. `ERROR` level log messages on the producer are likely to indicate that sending the message failed completely due to nonretriable error, a retriable error that ran out of retries, or a timeout. When applicable, the exact error from the broker will be logged as well.

On the consumer side, the most important metric is consumer lag. This metric indicates how far the consumer is from the latest message commit-

ted to the partition on the broker. Ideally, the lag would always be zero and the consumer will always read the latest message. In practice, because calling `poll()` returns multiple messages and then the consumer spends time processing them before fetching more messages, the lag will always fluctuate a bit. What is important is to make sure consumers do eventually catch up rather than fall further and further behind. Because of the expected fluctuation in consumer lag, setting traditional alerts on the metric can be challenging. [Burrow](#) is a consumer lag checker by LinkedIn and can make this easier.

Monitoring flow of data also means making sure all produced data is consumed in a timely manner (“timely manner” is usually based on business requirements). In order to make sure data is consumed in a timely manner, we need to know when the data was produced. Kafka assists in this: starting with version 0.10.0, all messages include a timestamp that indicates when the event was produced (although note that this can be overridden either by the application that is sending the events or by the brokers themselves if they are configured to do so).

To make sure all produced messages are consumed within a reasonable amount of time, we will need the application producing the messages to record the number of events produced (usually as events per second). The consumers need to record the number of events consumed per unit or time, and the lag from the time events were produced to the time they were consumed, using the event timestamp. Then we will need a system to reconcile the events per second numbers from both the producer and the consumer (to make sure no messages were lost on the way) and to make sure the interval between produce time and consume time is reasonable. This type of end-to-end monitoring systems can be challenging and time-consuming to implement. To the best of our knowledge, there is no open source implementation of this type of system, but Confluent provides a commercial implementation as part of the [Confluent Control Center](#).

In addition to monitoring clients and the end-to-end flow of data, Kafka brokers include metrics that indicate the rate of error responses sent from the brokers to clients. We recommend collecting

```
kafka.server:type=BrokerTopicMetrics,  
name=FailedProduceRequestsPerSec and  
kafka.server:type=BrokerTopic
```

`Metrics,name=FailedFetchRequestPerSec` . At times, some level of error responses is expected—for example, if we shut down a broker for maintenance and new leaders are elected on another broker, it is expected that producers will receive a `NOT_LEADER_FOR_PARTITION` error, which will cause them to request updated metadata before continuing to produce events as usual. Unexplained increases in failed requests should always be investigated. To assist in such investigations, the failed requests metrics are tagged with the specific error response that the broker sent.

Summary

As we said in the beginning of the chapter, reliability is not just a matter of specific Kafka features. We need to build an entire reliable system, including the application architecture, the way applications use the producer and Consumer APIs, producer and consumer configuration, topic configuration, and broker configuration. Making the system more reliable always has trade-offs in application complexity, performance, availability, or disk-space usage. By understanding all the options and common patterns and understanding requirements for each use case, we can make informed decisions regarding how reliable the application and Kafka deployment need to be and which trade-offs make sense.